UANL
UNIVERSIDAD AUTÓNOMA DE NUEVO LEÓN

UNIVERSIDAD AUTONOMA DE NUEVO LEON
Facultad de Ingenieria Mecanica y Electrica

FIME
FACULTAD DE INGENIERÍA MECÁNICA Y ELÉCTRICA

Sistemas Operativos

Actividad Fundamental 2: Multitarea y control de concurrencia

Equipo #2

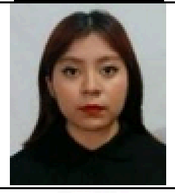
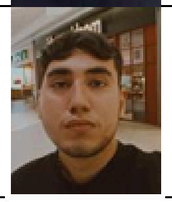
Docente: Dra. Norma Edith Marín Martínez

Hora: V4 – V6 Día: Martes Salón: 1303 Grupo: 003

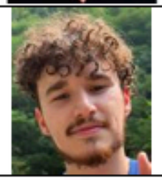
Fecha de Entrega: 1 de septiembre de 2026

Semestre: Agosto- Diciembre 2026

INTEGRANTES

	Matricula	Nombre	Carrera	Porcentaje
	2171232	Cruz Leyton Diego	IAS	100%
	2226958	Cotlame Nieto Lesly Adamaris	ITS	100%
	2160745	Del Ángel Martínez Edgar Jesús	ITS	100%
	2226766	Díaz García Josue Emiliano	ITS	100%
	2226678	Hernández Fraire Diana Lucia	ITS	100%

INTEGRANTES

	2019069	Juarez Luna Cristopher Led	ITS	100%
	2103101	Leal García Edgar Adan	IAS	100%
	2060308	Rodríguez Ortiz Danna Saraí	ITS	100%
	2148132	Santos Ramírez Edmundo	ITS	100%
	2153261	Torres Cabriales Aaron Sebastián	ITS	100%
	2101844	Torres Pasillas Ángela Susana	ITS	100%
	2169894	Ordaz García Valeria	ITS	100%

ÍNDICE

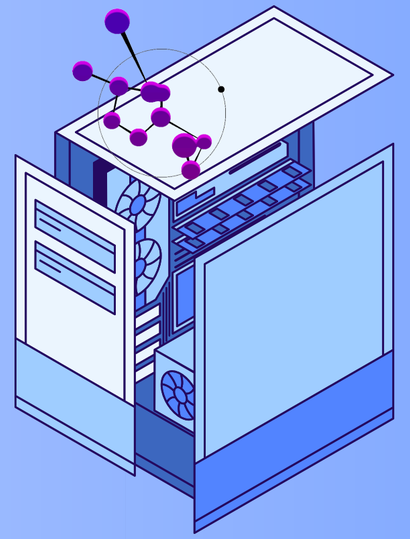
Introducción 5
¿Qué es la concurrencia? 6
Objetivos de la concurrencia 7
Fases de la concurrencia 8
Tipos de concurrencia 15
Modelos de programación concurrente 17
Ventajas de la ejecución concurrente 25
Tipos de procesos concurrentes 29
Tipos de interacciones entre los procesos dentro de la concurrencia 32
Paralelismo 39
Exclusión mutua y ejemplos 41

INTRODUCCIÓN

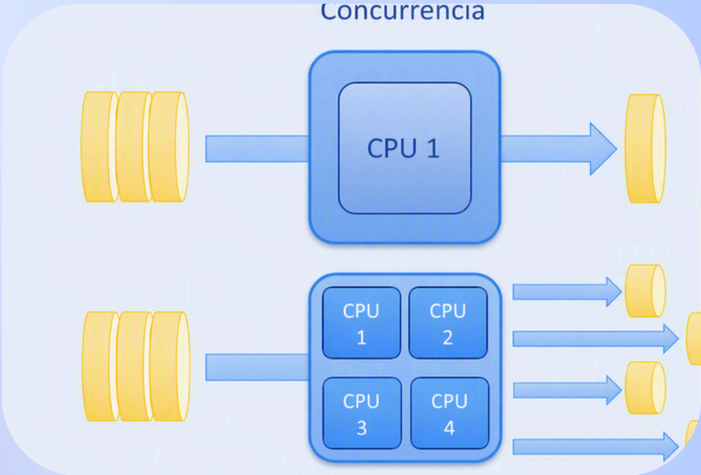
En la actualidad, las computadoras, los teléfonos celulares y otros dispositivos electrónicos forman parte de nuestra vida diaria y nos permiten realizar muchas actividades al mismo tiempo. Para que estos dispositivos funcionen de manera eficiente, los sistemas operativos administran los recursos del equipo y organizan la ejecución de los procesos. En este contexto, la concurrencia es un concepto fundamental, ya que permite ejecutar varias tareas de forma organizada y contribuye al buen funcionamiento del sistema. Comprender este concepto es importante para conocer cómo los sistemas operativos coordinan los procesos y mejoran el rendimiento de las aplicaciones que utilizamos todos los días.



¿QUÉ ES LA CONCURRENCIA?



La concurrencia es la capacidad de un sistema operativo para ejecutar dos o más procesos o hilos al mismo tiempo. En un solo núcleo, la CPU cambia rápidamente entre procesos, mientras que en varios núcleos estos pueden ejecutarse de forma simultanea.



OBJETIVOS DE LA CONCURRENCIA

Maximizar el uso de recursos y el rendimiento

El hardware (CPU, memoria, dispositivos de Entrada/Salida) es un recurso limitado y costoso. La concurrencia permite solapar el cómputo de la CPU con las operaciones de E/S. Mientras un proceso espera a que concluya una lectura/escritura en disco o red, la CPU se asigna a otro proceso activo, evitando tiempos muertos del procesador.

Aumentar la interactividad y la capacidad de respuesta

La concurrencia asegura que el usuario pueda seguir interactuando con la interfaz gráfica o las aplicaciones en primer plano (por ejemplo, escribir en un editor de texto) mientras el sistema procesa tareas extensas en segundo plano (por ejemplo, descargar un archivo o compilar código).

FASES DE LA CONCURRENCIA



LOS PROCESOS PUEDEN CAMBIAR DE ESTADO VARIAS VECES ANTES DE TERMINAR

FASES DE LA CONCURRENCIA

1 NUEVO

El proceso acaba de ser creado por el sistema operativo y está siendo preparado para ejecutarse.



Al abrir un programa como Google Chrome, el sistema operativo crea el proceso y lo prepara.

FASES DE LA CONCURRENCIA

2 LISTO

El proceso ya está preparado para ejecutarse, pero está esperando que el CPU esté disponible.

Tener abiertos Word, Excel y Spotify. Ya están listos para ejecutarse, pero esperan su turno para usar el CPU.



FASES DE LA CONCURRENCIA

3 EJECUCIÓN

El proceso está utilizando el CPU, que se encuentra ejecutando sus instrucciones.



Estar escribiendo en Microsoft Word. El CPU está ejecutando las instrucciones para mostrar lo que escribes.

FASES DE LA CONCURRENCIA

4 BLOQUEADO O EN ESPERA

El proceso no puede continuar temporalmente porque está esperando un recurso o que ocurra algún evento.

Un navegador espera que cargue una página desde Internet. Mientras tanto, queda en espera.



FASES DE LA CONCURRENCIA

5 TERMINADO

El proceso ha terminado su ejecución y el sistema operativo libera los recursos que estaba utilizando.



Cierras un programa o termina su tarea. El sistema operativo finaliza el proceso y libera la memoria.

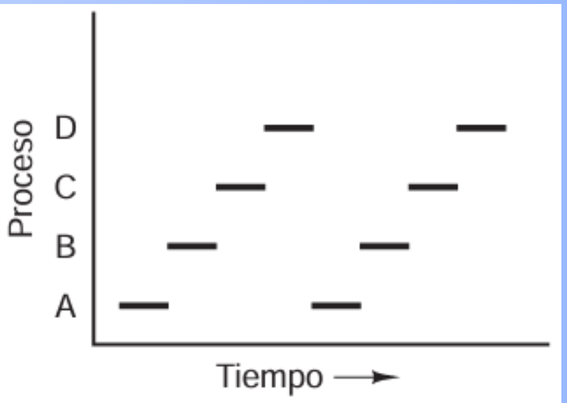
FASES DE LA CONCURRENCIA

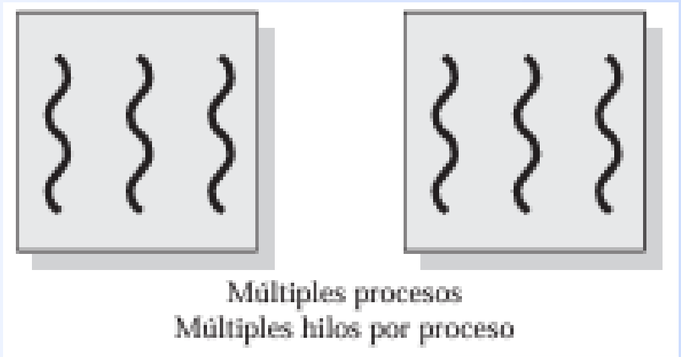


TIPOS DE CONCURRENCIA

Concurrencia entre procesos

Ocurre cuando dos o más procesos se ejecutan durante el mismo periodo de tiempo. Cada proceso tiene su propio espacio de memoria, el sistema operativo debe adiministrar cuándo y cuánto tiempo utiliza cada uno.





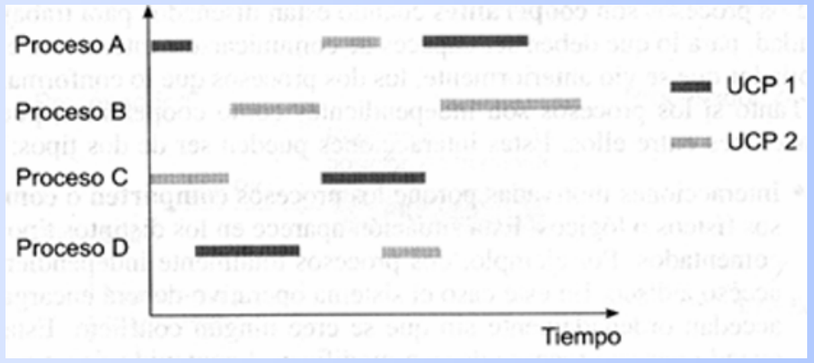
Concurrencia entre hilos

Se presenta cuando varios hilos pertenecientes a un mismo proceso se ejecutan de forma concurrente. Los hilos comparten una misma memoria, lo que permite que la comunicación entre ellos sea mas rápida. Sin embargo, existe el riesgo de errores si no se coordinan correctamente.

TIPOS DE CONCURRENCIA

Concurrencia en un solo procesador

La CPU cambia muy rapidamente de un proceso o hilo a otro. Realmente solo se ejecuta uno a al vez, la velocidad con la que ocurre este cambio da la impresión de que varios procesos se estan ejecutando al mismo tiempo.



Ej. de concurrencia en varios procesadores

MODELOS DE PROGRAMACIÓN CONCURRRENTE



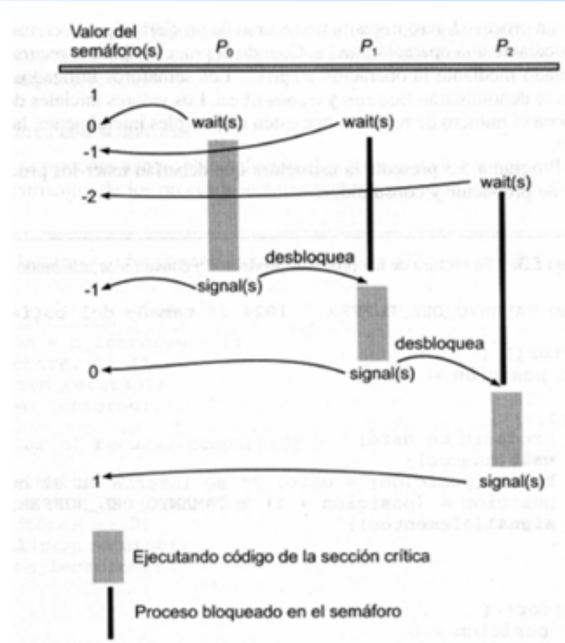
Semáforos

Mecanismos de sincronización que controlan el acceso a recursos compartidos mediante las operaciones wait y signal. Permiten coordinar la ejecución de procesos, evitar conflictos en la sección crítica y pueden ser binarios o contadores.

Sección crítica

- Controlan el acceso a recursos compartidos.
- Evitan que varios procesos entren simultáneamente.
- wait(): solicita acceso.
- signal(): libera el recurso.
- Semáforo inicializado en 1 → acceso exclusivo.

Funcionamiento:



Problema lectores-escritores

- Existe un recurso compartido.
- Varios lectores pueden acceder simultáneamente.
- Un escritor necesita acceso exclusivo.

Se utilizan dos semáforos:

- sem_recurso
- sem_lectores

```
Lector( ) {
      wait (sem_lectores);
      int_lectores = n_lectores + 1;
      if (n_lectores = = 1)
        wait (sem_recurso);
        signal (sem-lectores);

        < consultar el recurso compartido >

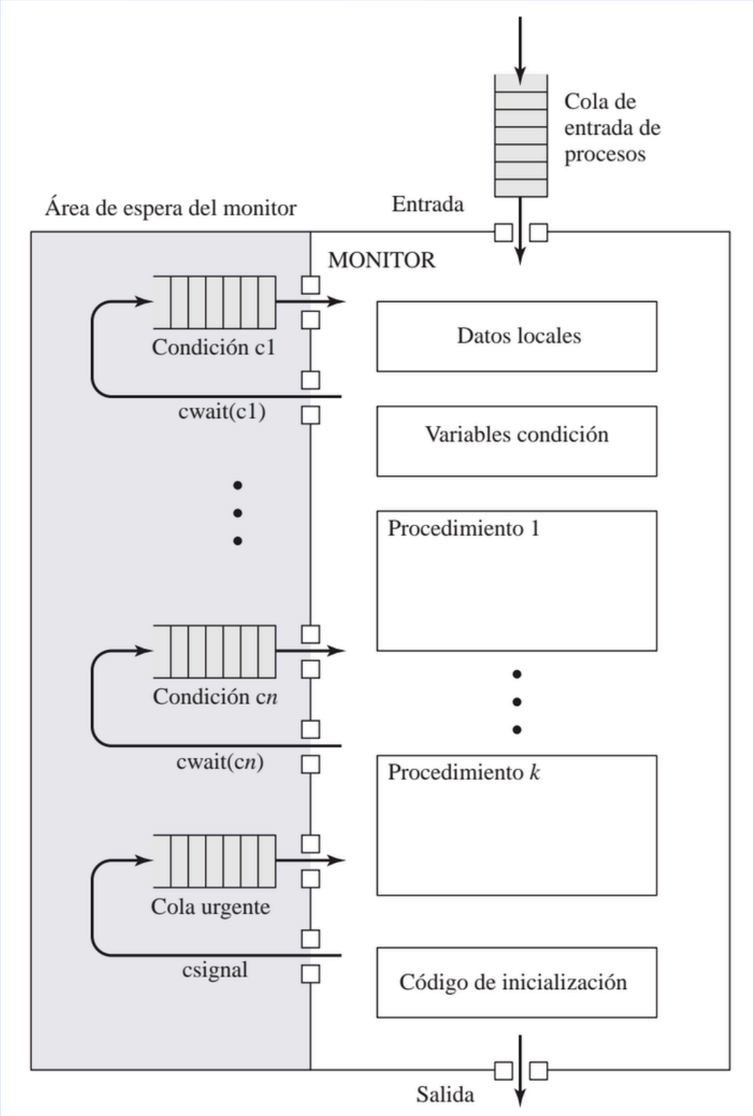
        wait (sem_lectores);
        n_lectores = n_lectores _ 1;
        if (n-lectores = = 0)
            signal (sen_recurso);
        signal(sem-lectores);
}
 Escritor ( ) {
      wait (sem_recurso);
      /* se puede modificar el recurso */
      signal(sem_recurso);
}
```

Monitores

Los monitores son una herramienta para controlar el acceso de varios procesos a recursos compartidos.

- Agrupan los datos y procedimientos relacionados con un recurso.
- Los datos del monitor no pueden ser modificados directamente desde fuera.
- Solo un proceso puede estar dentro del monitor a la vez, mientras los demás esperan.
- Ayudan a mantener la exclusión mutua y a coordinar la ejecución de los procesos.
- Utilizan variables de condición para que un proceso pueda esperar hasta que se cumpla determinada situación.
- Sus operaciones principales son cwait, que pone a esperar un proceso, y csignal, que permite continuar a uno que estaba esperando.

Estructura de un Monitor

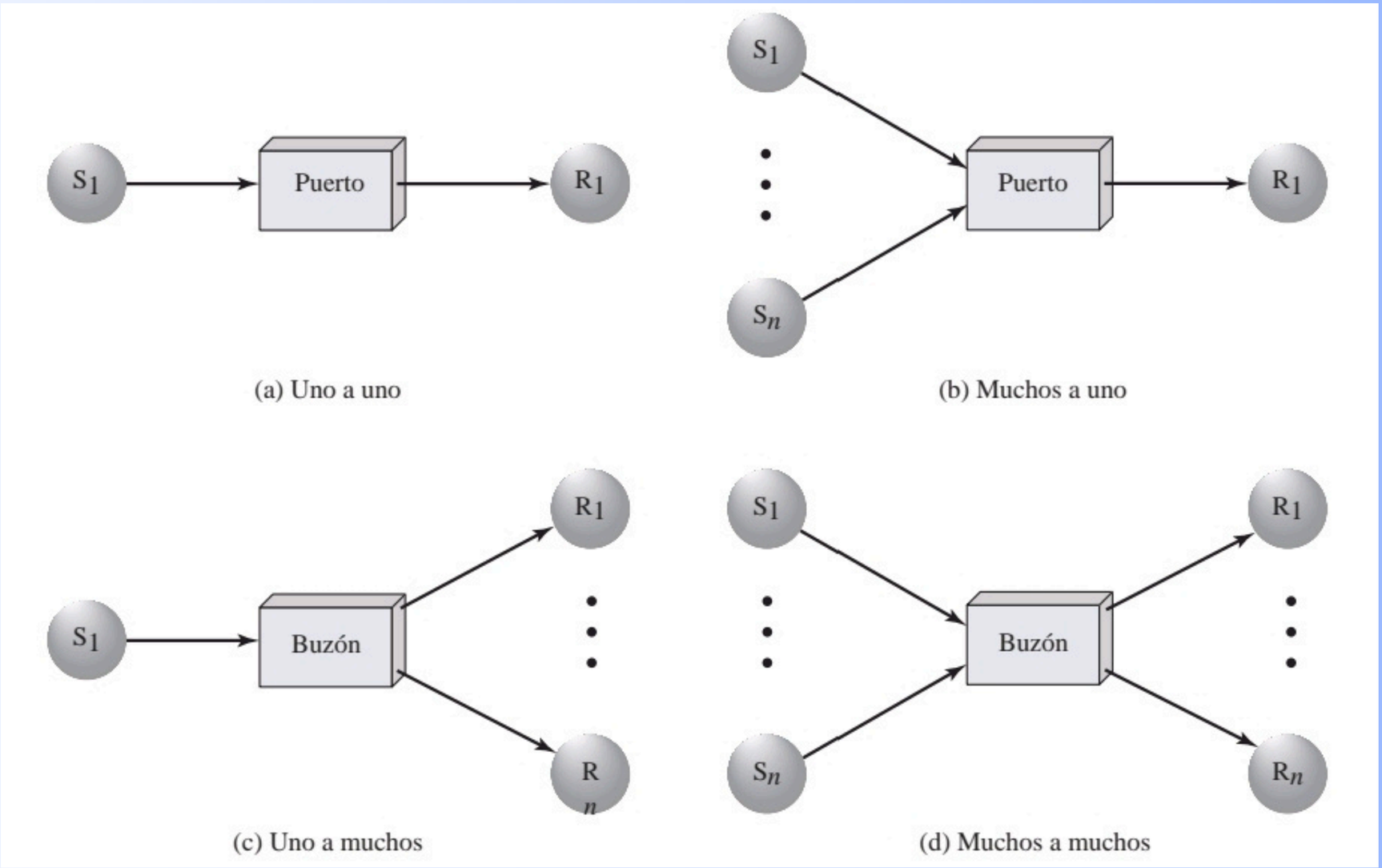


Paso de Mensajes

El paso de mensajes permite que diferentes procesos se comuniquen e intercambien información, además de ayudar a coordinar su ejecución.

- Send: permite enviar un mensaje a otro proceso.
- Receive: permite recibir un mensaje de otro proceso.
- Puede utilizarse para comunicación y sincronización.
- Las operaciones pueden ser bloqueantes o no bloqueantes.
- Puede utilizar buzones para almacenar y organizar los mensajes.
-También puede utilizarse para controlar la exclusión mutua y resolver problemas como productor/consumidor.

Comunicación indirecta de procesos



VENTAJAS DE LA EJECUCIÓN CONCURRENTE



1. Mayor rendimiento del sistema

Permite que varios procesos o hilos avancen al mismo tiempo, logrando que el sistema operativo complete más tareas en menos tiempo. Esto mejora el rendimiento de la computadora, especialmente cuando se ejecutan varias aplicaciones de forma simultánea.

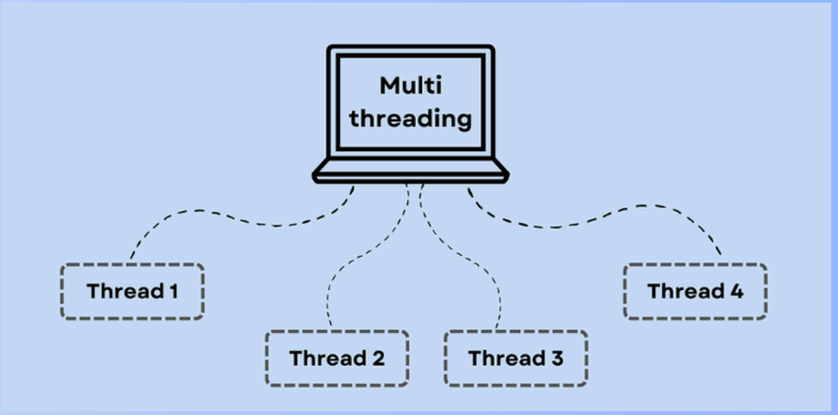


VENTAJAS DE LA EJECUCIÓN CONCURRENTE



2. Mejor capacidad de respuesta (Responsiveness)

Las aplicaciones pueden seguir respondiendo al usuario mientras realizan otras tareas en segundo plano, evitando bloqueos y ofreciendo una experiencia de uso más rápida y fluida.

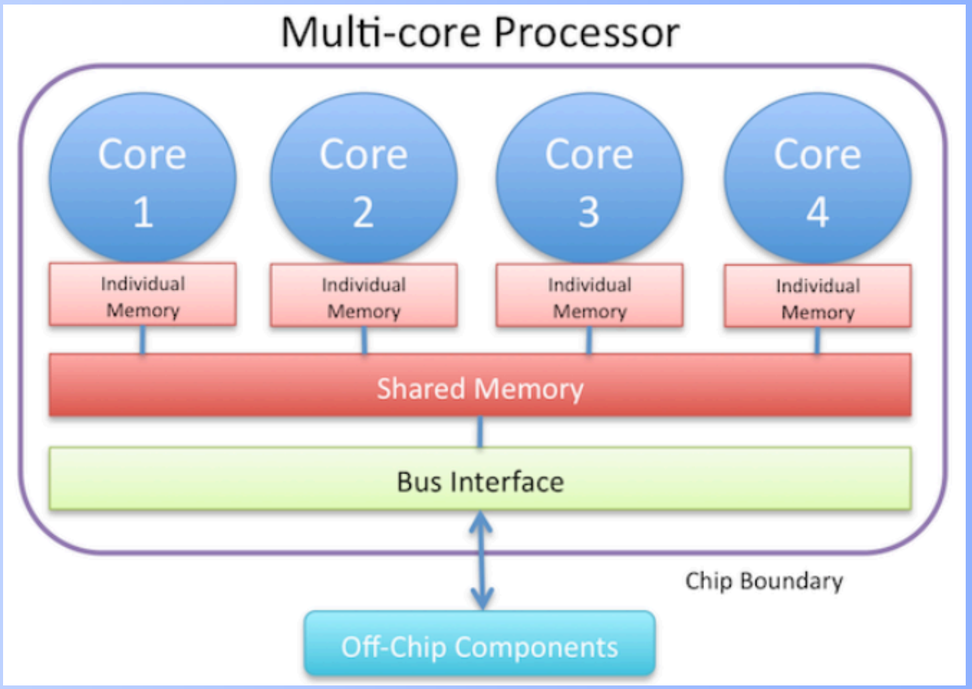


VENTAJAS DE LA EJECUCIÓN CONCURRENTE



3. Mayor escalabilidad (Scalability)

Aprovecha mejor los procesadores con varios núcleos al distribuir las tareas entre ellos. Además, permite que el sistema maneje más procesos o usuarios sin disminuir significativamente su rendimiento.

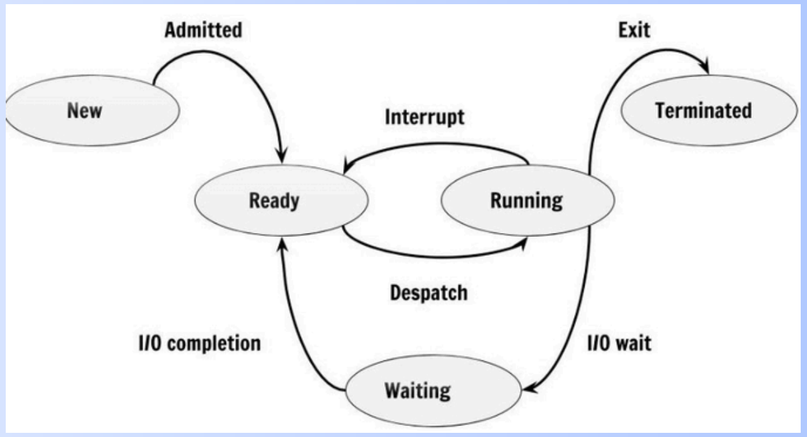


VENTAJAS DE LA EJECUCIÓN CONCURRENTE



4. Uso eficiente de los recursos del sistema

Optimiza el uso del procesador, la memoria y los dispositivos de entrada y salida. Mientras un proceso espera una operación, el sistema puede ejecutar otro, evitando que los recursos permanezcan inactivos.



TIPOS DE

PROCESOS CONCURRENTES



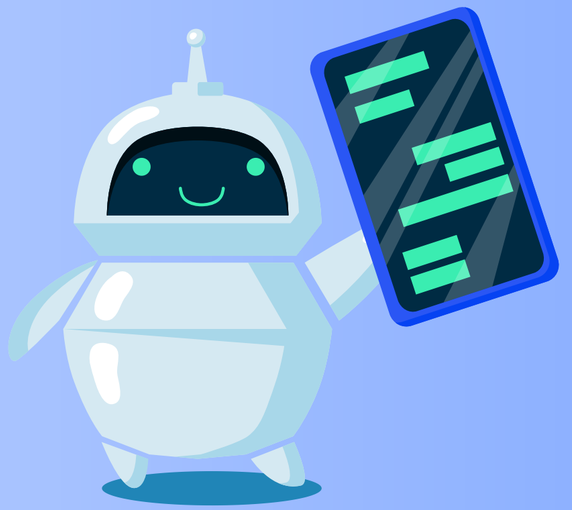
En los sistemas operativos, los procesos que se ejecutan de forma concurrente se pueden clasificar principalmente en dos tipos según la naturaleza de su ejecución



Procesos Independientes

Se ejecutan sin requerir ayuda o cooperación de otros.
Son totalmente autónomos; su avance no depende de las demás tareas.
Ejemplo: Varios intérpretes de mandatos (consolas) ejecutándose al mismo tiempo sin interactuar.

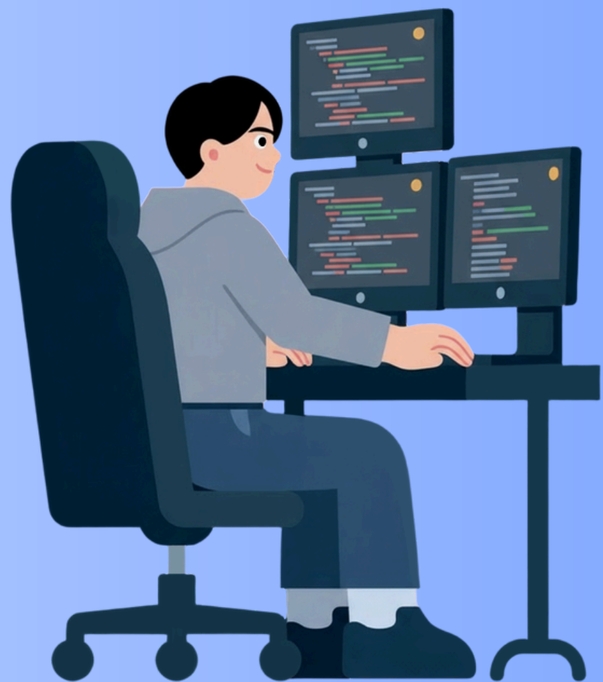




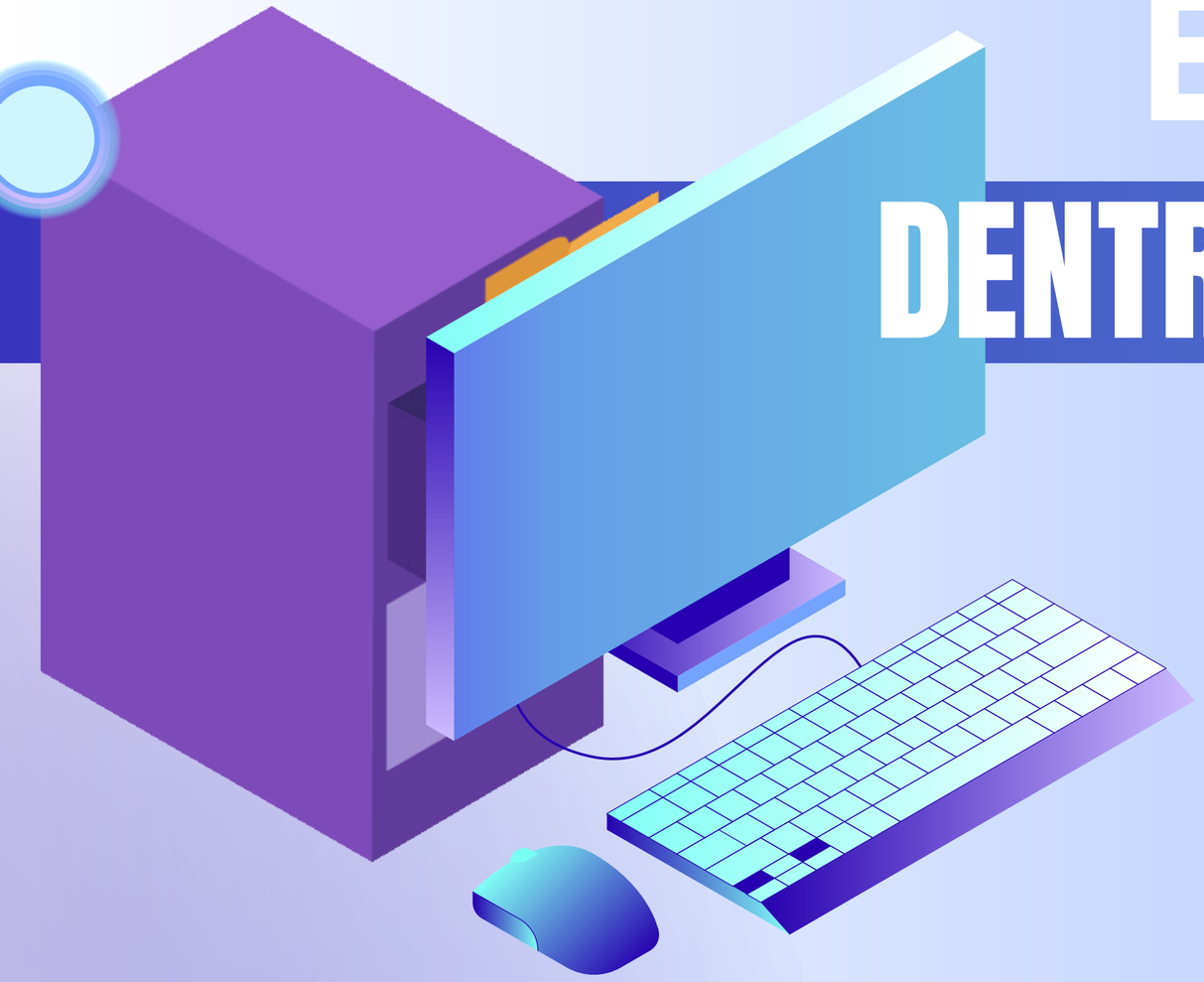
Procesos Cooperativos

Están diseñados para trabajar en conjunto en una misma actividad.
Necesitan comunicarse e interactuar entre sí durante su ejecución.
Ejemplo: Un compilador, donde procesos como el compilador y el ensamblador se sincronizan para crear código máquina.





TIPOS DE INTERACCIONES ENTRE LOS PROCESOS DENTRO DE LA CONCURRENCIA



En un sistema operativo, se pueden ejecutar distintos procesos, a la misma vez o alternadas. Durante su ejecucion puede haber diferentes manera de interacción, esto ya depende de si comparten informacion o recursos.

INTERACCION POR INDEPENDENCIA

Este tipo de interacción sucede cuando dos o mas procesos se pueden ejecutar de manera autónoma, ya que no necesita de información u otros tipos de datos para realizar sus tareas.

Esto ocurre porque cada proceso mantiene su propio espacio de trabajo y no tienen necesidad de comunicarse con otros procesos



INTERACCIONES PARA LA TRANSMICIÓN



Esta interacion ocurre cuando dos o más procesos requieren pasarse información o datos entre si para llegar a una meta en común.

Los procesos interactúan entre si enviando y recibiendo informacion además de tener un espacio de memoria común al que todos los procesos involucrados tienen acceso directo.

INTERACCIONES POR COMPETENCIA



Este tipo de interacción esta presente cuando dos o mas procesos compiten de manera simultanea po el mismo recurso finito del sistema.

Es una situación problematica en el que los procesos incolucrados leen o escriben los mismos datos compartidos.

Existe el mecanismo de “exclusión mutua” es un mecanismo que evita que otros procesos queden excluidos de cierto recurso compartido si un proceso ya lo esta utilizando.

INTERACCIONES PARA LA SINCRONIZACION Y COORDINACIÓN



Esta interaccion ocurre cuando las acciones o el avance de un proceso depende del progreso de otro proceso:

Algunas maneras de evitar que un proceso este estancado mucho rato son aplicando:

- Dormir y despertar
- Semáforos y mutexes
- Barreras

INTERACCIONES QUE GENERAN CONFLICTO

Existen situaciones donde la interacción entre procesos ya no se pueden llevar a cabo, esto debido a que cuando dos o mas procesos quedan bloqueados de manera permanente porque cada uno de los procesos sostiene un recurso que el otro proceso necesita para seguir su ejecución, a esto se le conoce como “Interbloqueos”





INTERACCION POR COMUNICACIÓN

Ocurre cuando dos o más procesos que cooperan necesitan intercambiar información para poder trabajar de manera conjunta.

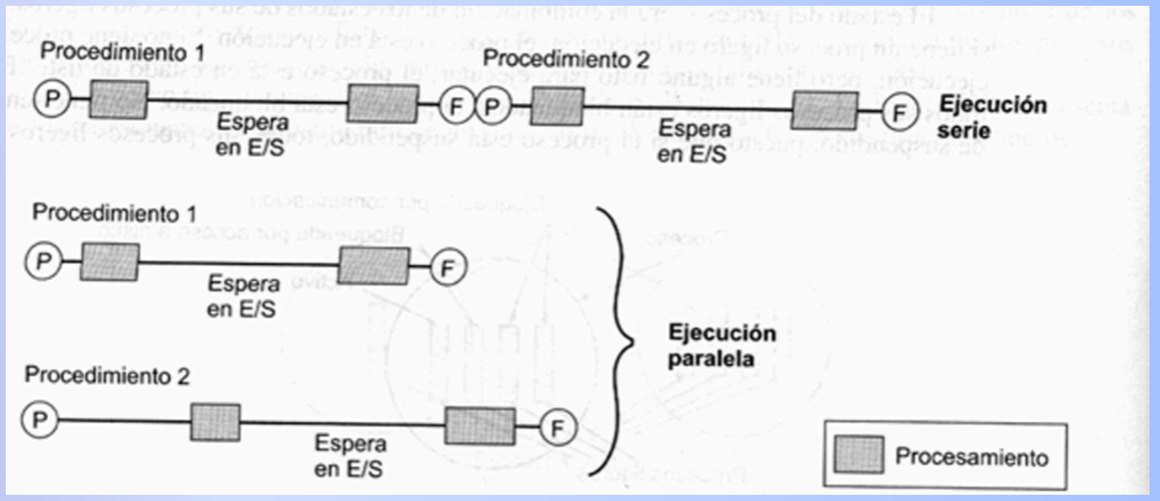
Al ser procesos cooperativos su ejecución depende de los datos proporcionados por los demás procesos.

Además de tener un mecanismo de comunicación que permite que los procesos puedan enviar y recibir datos durante la ejecución.

PARALELISMO

El paralelismo consta en la ejecución simultánea de dos o más hilos/procesos utilizando múltiples núcleos de CPU.

Se busca reducir el tiempo de ejecución de varios hilos/procesos utilizando núcleos de CPU dependiendo de la cantidad de hilos/procesos que se encuentren.



CONCURRENCIA VS PARALELISMO

PARALELISMO

- El paralelismo es un modelo de ejecución.
- Se enfoca en realizar múltiples tareas simultáneamente.

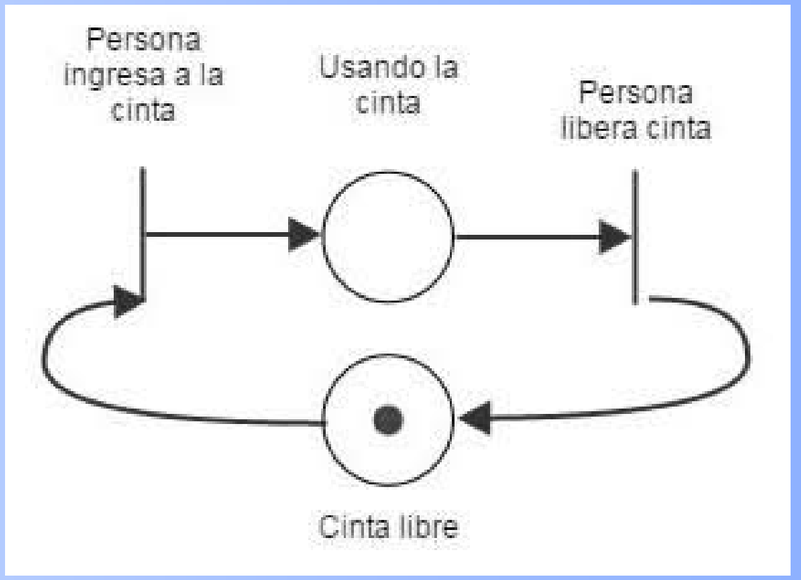
CONCURRENCIA

- La concurrencia es un modelo de organización.
- Se enfoca en organizar diferentes tareas para poder lidear con todas ellas a la vez.

Exclusión mutua

Es un mecanismo que se usa principalmente en los sistemas operativos y en la programación concurrente esto para controlar el acceso de varios procesos a un recurso compartido.

Tiene como objetivo garantizar que solo un proceso pueda utilizar el recurso, mientras que los demás procesos deben de esperarse para poder usar ese mismo recurso.



EJEMPLOS EN LA VIDA DIARIA DE LA EXCLUSIÓN MUTUA

Una Impresora

Cuando hay varias impresiones por adelante, los documentos deben de ser procesados de una manera en que uno termine de ser impreso para que el siguiente documento siga.





Cajero automático

cuando una persona esta realizando una operación en el cajero, la siguiente persona debe de esperar su turno para usar ese mismo cajero.

CONCLUSIÓN GENERAL

Como equipo, concluimos que la concurrencia es una parte muy importante de los sistemas operativos, ya que permite realizar varias tareas y aprovechar mejor los recursos de la computadora. A través de este trabajo comprendimos los diferentes tipos de concurrencia, los modelos de programación y la forma en que los procesos pueden comunicarse y trabajar juntos.

También aprendimos que, aunque la concurrencia mejora el rendimiento y la respuesta del sistema, es necesario coordinar correctamente los procesos para evitar problemas como errores, condiciones de carrera o interbloqueos. En general, este tema nos ayudó a entender mejor cómo los sistemas operativos organizan las tareas que realizamos diariamente en nuestros dispositivos.

BIBLIOGRAFÍA

- Carretero Pérez, J., García Carballeira, F., de Miguel Anasagasti, P., & Pérez Costoya, F. (2001). Sistemas operativos: Una visión aplicada (1.ª ed.). McGraw-Hill.
- Tanenbaum, Andrew S. (2009). Sistemas Operativos Modernos (3ª ed.). Pearson Educación.
- Stallings, W. (2005). Sistemas operativos: Aspectos internos y principios de diseño (5.ª ed.). Pearson Educación.
- Wolf, G., Ruiz, E., Bergero, F., & Meza, E. (2015). Fundamentos de sistemas operativos. Universidad Nacional Autónoma de México, Instituto de Investigaciones Económicas, Facultad de Ingeniería.
- Silberschatz, A., Galvin, P. B., & Gagne, G. (2014). Sistemas operativos (9.ª ed.). McGraw-Hill Education.